4.1 Exercise on Functions:

Task - 1: Create a Python program that converts between different units of measurement. • The program should:

1. Prompt the user to choose the type of conversion (e.g., length, weight, volume).
2. Ask the user to input the value to be converted.
3. Perform the conversion and display the result.
4. Handle potential errors, such as invalid input or unsupported conversion types. • Requirements:
5. Functions: Define at least one function to perform the conversion.
6. Error Handling: Use try-except blocks to handle invalid input (e.g., non-numeric values).
7. User Input: Prompt the user to select the conversion type and input the value.
8. Docstrings: Include a docstring in your function to describe its purpose, parameters, and return value. • Conversion Options:
9. Length: – Convert meters (m) to feet (ft). – Convert feet (ft) to meters (m).
10. Weight: – Convert kilograms (kg) to pounds (lbs). – Convert pounds (lbs) to kilograms (kg).
11. Volume: – Convert liters (L) to gallons (gal). – Convert gallons (gal) to liters (L).

```
def convert_units(conversion_type, value):
    conversions = {
        "m_to_ft": value * 3.28084,
        "ft_to_m": value / 3.28084,
        "kg_to_lbs": value * 2.20462,
        "lbs_to_kg": value / 2.20462,
        "l_to_gal": value * 0.264172,
        "gal_to_l": value / 0.264172
    }

    if conversion_type in conversions:
        return conversions[conversion_type]
    else:
        raise ValueError("Unsupported conversion type.")

def unit_conversion_program():
    print("Select conversion type:")
    print("1. m_to_ft")
    print("2. ft_to_m")
    print("3. kg_to_lbs")
    print("4. lbs_to_kg")
    print("5. l_to_gal")
    print("6. gal_to_l")
```

```
    choice_map = {
        "1": "m_to_ft",
        "2": "ft_to_m",
        "3": "kg_to_lbs",
        "4": "lbs_to_kg",
        "5": "l_to_gal",
        "6": "gal_to_l"
    }

    try:
        choice = input("Enter choice (1-6): ")
        if choice not in choice_map:
            raise ValueError("Invalid choice.")

        value = float(input("Enter value to convert: "))
        result = convert_units(choice_map[choice], value)

        print(f"Converted value: {result}")

    except ValueError as e:
        print("Error:", e)

unit_conversion_program()
```

```
Select conversion type:
1. m_to_ft
2. ft_to_m
3. kg_to_lbs
4. lbs_to_kg
5. l_to_gal
6. gal_to_l
Enter choice (1-6): 6
Enter value to convert: 50
Converted value: 189.27062671289917
```

Task - 2:

Create a Python program that performs various mathematical operations on a list of numbers. • The Program should:

1. Prompt the user to choose an operation (e.g., find the sum, average, maximum, or minimum of the numbers).
2. Ask the user to input a list of numbers (separated by spaces).
3. Perform the selected operation and display the result.

4. Handle potential errors, such as invalid input or empty lists. • Requirements:
5. Functions: Define at least one function for each operation (sum, average, maximum, mini- mum).
6. Error Handling: Use try-except blocks to handle invalid input (e.g., non-numeric values or empty lists).
7. User Input: Prompt the user to select the operation and input the list of numbers.
8. Docstrings: Include a docstring in each function to describe its purpose, parameters, and return value.

```
def calculate_sum(numbers):
    return sum(numbers)

def calculate_average(numbers):
    if len(numbers) == 0:
        raise ValueError("List is empty.")
    return sum(numbers) / len(numbers)

def find_maximum(numbers):
    if len(numbers) == 0:
        raise ValueError("List is empty.")
    return max(numbers)

def find_minimum(numbers):
    if len(numbers) == 0:
        raise ValueError("List is empty.")
    return min(numbers)

def math_operations_program():
    print("Choose operation:")
    print("1. Sum")
    print("2. Average")
    print("3. Maximum")
    print("4. Minimum")

    try:
        choice = input("Enter choice (1-4): ")
        numbers = list(map(float, input("Enter numbers separated by spaces: ").split()))

        if not numbers:
            raise ValueError("List cannot be empty.")

        if choice == "1":
            print("Result:", calculate_sum(numbers))
        elif choice == "2":
            print("Result:", calculate_average(numbers))
```

```
        elif choice == "3":
            print("Result:", find_maximum(numbers))
        elif choice == "4":
            print("Result:", find_minimum(numbers))
        else:
            print("Invalid choice.")

    except ValueError:
        print("Error: Please enter valid numeric input.")

math_operations_program()
```

```
Choose operation:
1. Sum
2. Average
3. Maximum
4. Minimum
Enter choice (1-4): 1
Enter numbers separated by spaces: 5 6
Result: 11.0
```

4.2 Exercise on List Manipulation:

1. Extract Every Other Element: Write a Python function that extracts every other element from a list, starting from the first element.
2. Slice a Sublist: Write a Python function that returns a sublist from a given list, starting from a specified index and ending at another specified index.
3. Reverse a List Using Slicing: Write a Python function that reverses a list using slicing.
4. Remove the First and Last Elements: Write a Python function that removes the first and last elements of a list and returns the resulting sublist.
5. Get the First n Elements: Write a Python function that extracts the first n elements from a list.
6. Extract Elements from the End: Write a Python function that extracts the last n elements of a list using slicing.
7. Extract Elements in Reverse Order: Write a Python function that extracts a list of elements in reverse order starting from the second-to-last element and skipping one element in between.

```
def extract_every_other(lst):
    return lst[::2]

def get_sublist(lst, start, end):
```

```
    return lst[start:end+1]

def reverse_list(lst):
    return lst[::-1]

def remove_first_last(lst):
    return lst[1:-1]

def get_first_n(lst, n):
    return lst[:n]

def get_last_n(lst, n):
    return lst[-n:]

def reverse_skip(lst):
    return lst[-2::-2]

data = [10, 21, 32, 43, 54, 65, 76, 87, 98, 109]

every_other = extract_every_other(data)
first_five = get_first_n(data, 5)
last_three = get_last_n(data, 3)
middle_part = remove_first_last(data)
sub = get_sublist(middle_part, 1, 4)
reversed_sub = reverse_list(sub)
weird_backward = reverse_skip(data)

print("Original:         ", data)
print("Every other:      ", every_other)
print("First 5:          ", first_five)
print("Last 3:           ", last_three)
print("Without first & last:", middle_part)
print("Sublist(1:4) from middle:", sub)
print("Reversed sublist: ", reversed_sub)
print("reverse_skip:     ", weird_backward)
```

```
Original:         [10, 21, 32, 43, 54, 65, 76, 87, 98, 109]
Every other:      [10, 32, 54, 76, 98]
First 5:          [10, 21, 32, 43, 54]
Last 3:           [87, 98, 109]
Without first & last: [21, 32, 43, 54, 65, 76, 87, 98]
Sublist(1:4) from middle: [32, 43, 54, 65]
Reversed sublist:  [65, 54, 43, 32]
reverse_skip:     [98, 76, 54, 32, 10]
```

4.3 Exercise on Nested List:

1. Flatten a Nested List: Write a Python function that takes a nested list and flattens it into a single list, where all the elements are in a single dimension.
2. Accessing Nested List Elements: Write a Python function that extracts a specific element from a nested list given its indices.
3. Sum of All Elements in a Nested List: Write a Python function that calculates the sum of all the numbers in a nested list (regardless of depth).
4. Remove Specific Element from a Nested List: Write a Python function that removes all occurrences of a specific element from a nested list.
5. Find the Maximum Element in a Nested List: Write a Python function that finds the maximum element in a nested list (regardless of depth)
6. Count Occurrences of an Element in a Nested List: Write a Python function that counts how many times a specific element appears in a nested list.
7. Flatten a List of Lists of Lists: Write a Python function that flattens a list of lists of lists into a single list, regardless of the depth.
8. Nested List Average: Write a Python function that calculates the average of all elements in a nested list.

```
def flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(item)
        else:
            result.append(item)
    return result

def access_nested_element(lst, indices):
    element = lst
    for index in indices:
        element = element[index]
    return element

def sum_nested(lst):
    total = 0
    for item in lst:
        if isinstance(item, list):
            total += sum_nested(item)
        else:
```

```
            total += item
    return total

def remove_element(lst, elem):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.append(remove_element(item, elem))
        elif item != elem:
            result.append(item)
    return result

def find_max(lst):
    maximum = float('-inf')
    for item in lst:
        if isinstance(item, list):
            maximum = max(maximum, find_max(item))
        else:
            maximum = max(maximum, item)
    return maximum

def count_occurrences(lst, elem):
    count = 0
    for item in lst:
        if isinstance(item, list):
            count += count_occurrences(item, elem)
        elif item == elem:
            count += 1
    return count

def deep_flatten(lst):
    result = []
    for item in lst:
        if isinstance(item, list):
            result.extend(deep_flatten(item))
        else:
            result.append(item)
    return result

def average_nested(lst):
    flat_list = deep_flatten(lst)
    if len(flat_list) == 0:
        raise ValueError("List is empty.")
    return sum(flat_list) / len(flat_list)

nested = [1, 2, [3, 4, [5, 5, 6]], [7, [8, 9]], 10, [11, [12]], 5]
```

```
flat_one = flatten(nested)
print("flatten (one level):", flat_one)

val = access_nested_element(nested, [2, 2, 1])
print("access_nested_element([2,2,1]):", val)

total = sum_nested(nested)
print("sum_nested:", total)

cleaned = remove_element(nested, 5)
print("remove_element 5:", cleaned)

mx = find_max(nested)
print("find_max:", mx)

five_count = count_occurrences(nested, 5)
print("count_occurrences of 5:", five_count)

completely_flat = deep_flatten(nested)
print("deep_flatten:", completely_flat)

avg = average_nested(nested)
print("average_nested:", avg)
```

```
flatten (one level): [1, 2, 3, 4, [5, 5, 6], 7, [8, 9], 10, 11, [12], 5]
access_nested_element([2,2,1]): 5
sum_nested: 88
remove_element 5: [1, 2, [3, 4, [6]], [7, [8, 9]], 10, [11, [12]]]
find_max: 12
count_occurrences of 5: 3
deep_flatten: [1, 2, 3, 4, 5, 5, 6, 7, 8, 9, 10, 11, 12, 5]
average_nested: 6.285714285714286
```

Problem - 1: Array Creation:

Complete the following Tasks:

1. Initialize an empty array with size 2X2.
2. Initialize an all one array with size 4X2.
3. Return a new array of given shape and type, filled with fill value.{Hint: np.full}
4. Return a new array of zeros with same shape and type as a given array.{Hint: np.zeros like}
5. Return a new array of ones with same shape and type as a given array.{Hint: np.ones like}

6. For an existing list new_list = [1,2,3,4] convert to an numpy array.{Hint: np.array()}

```
import numpy as np
```

```
# 1. Initialize an empty array with size 2X2
empty_array = np.empty((2, 2))
print("Empty 2x2 array:")
print(empty_array)

# 2. Initialize an all ones array with size 4X2
ones_array = np.ones((4, 2))
print("\nAll-ones 4x2 array:")
print(ones_array)

# 3. Array filled with a fill value (e.g., fill_value=7, shape 3x3)
full_array = np.full((3, 3), fill_value=7)
print("\n np.full array (3x3, fill_value=7):")
print(full_array)

# 4. Array of zeros with same shape and type as a given array
given_array = np.array([[1, 2, 3], [4, 5, 6]])
zeros_like_array = np.zeros_like(given_array)
print("\n np.zeros_like (same shape as given_array):")
print(zeros_like_array)

# 5. Array of ones with same shape and type as a given array
ones_like_array = np.ones_like(given_array)
print("\n np.ones_like (same shape as given_array):")
print(ones_like_array)

# 6. Convert existing list to numpy array
new_list = [1, 2, 3, 4]
numpy_array = np.array(new_list)
print("\n Converted list to numpy array:")
print(numpy_array)
```

```
Empty 2x2 array:
[[4.9e-324 9.9e-324]
 [1.5e-323 2.0e-323]]

All-ones 4x2 array:
[[1. 1.]
 [1. 1.]
 [1. 1.]
```

```
 [1. 1.]]

 np.full array (3x3, fill_value=7):
[[7 7 7]
 [7 7 7]
 [7 7 7]]

 np.zeros_like (same shape as given_array):
[[0 0 0]
 [0 0 0]]

 np.ones_like (same shape as given_array):
[[1 1 1]
 [1 1 1]]

 Converted list to numpy array:
[1 2 3 4]
```

Problem - 2: Array Manipulation: Numerical Ranges and Array indexing:

Complete the following tasks:

1. Create an array with values ranging from 10 to 49. {Hint:np.arrange()}.
2. Create a 3X3 matrix with values ranging from 0 to 8. {Hint:look for np.reshape()}
3. Create a 3X3 identity matrix.{Hint:np.eye()}
4. Create a random array of size 30 and find the mean of the array. {Hint:check for np.random.random() and array.mean() function}
5. Create a 10X10 array with random values and find the minimum and maximum values.
6. Create a zero array of size 10 and replace 5th element with 1.
7. Reverse an array arr = [1,2,0,0,4,0].
8. Create a 2d array with 1 on border and 0 inside.
9. Create a 8X8 matrix and fill it with a checkerboard pattern.

```
# 1. Array from 10 to 49
arr = np.arange(10,50)
print("10 to 49:\n", arr)

# 2. 3x3 matrix 0 to 8
matrix = np.arange(9).reshape(3,3)
print("\n3x3 matrix:\n", matrix)

# 3. Identity matrix
identity = np.eye(3)
```

```
print("\nIdentity matrix:\n", identity)

# 4. Random array size 30 and mean
rand_arr = np.random.random(30)
print("\nMean:", rand_arr.mean())

# 5. 10x10 random min and max
rand_matrix = np.random.random((10,10))
print("\nMin:", rand_matrix.min())
print("Max:", rand_matrix.max())

# 6. Zero array size 10 and replace 5th element
z = np.zeros(10)
z[4] = 1
print("\nReplace 5th element:\n", z)

# 7. Reverse array
arr = np.array([1,2,0,0,4,0])
print("\nReversed:", arr[::-1])

# 8. 2D array with border 1 and inside 0
border = np.ones((5,5))
border[1:-1,1:-1] = 0
print("\nBorder array:\n", border)

# 9. 8x8 checkerboard
checker = np.zeros((8,8))
checker[::2,1::2] = 1
checker[1::2,::2] = 1
print("\nCheckerboard:\n", checker)
```

```
10 to 49:
 [10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49]

3x3 matrix:
 [[0 1 2]
 [3 4 5]
 [6 7 8]]

Identity matrix:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

Mean: 0.5826915714019937
```

```
Min: 0.0185249081617892
Max: 0.9848334791623026

Replace 5th element:
 [0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]

Reversed: [0 4 0 0 2 1]

Border array:
 [[1. 1. 1. 1. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 0. 0. 0. 1.]
 [1. 1. 1. 1. 1.]]

Checkerboard:
 [[0. 1. 0. 1. 0. 1. 0. 1.]
 [1. 0. 1. 0. 1. 0. 1. 0.]
 [0. 1. 0. 1. 0. 1. 0. 1.]
 [1. 0. 1. 0. 1. 0. 1. 0.]
 [0. 1. 0. 1. 0. 1. 0. 1.]
 [1. 0. 1. 0. 1. 0. 1. 0.]
 [0. 1. 0. 1. 0. 1. 0. 1.]
 [1. 0. 1. 0. 1. 0. 1. 0.]]
```

Problem - 3: Array Operations:

For the following arrays: x = np.array([[1,2],[3,5]]) and y = np.array([[5,6],[7,8]]); v = np.array([9,10]) and w = np.array([11,12]); Complete all the task using numpy:

1. Add the two array.
2. Subtract the two array.
3. Multiply the array with any integers of your choice.
4. Find the square of each element of the array.
5. Find the dot product between: v(and)w ; x(and)v ; x(and)y.
6. Concatenate x(and)y along row and Concatenate v(and)w along column. {Hint:try np.concatenate() or np.vstack() functions.
7. Concatenate x(and)v; if you get an error, observe and explain why did you get the error?

```
x = np.array([[1,2],[3,5]])
y = np.array([[5,6],[7,8]])
v = np.array([9,10])
w = np.array([11,12])
```

```
# 1. Add
print("Addition:\n", x + y)

# 2. Subtract
print("Subtraction:\n", x - y)

# 3. Multiply by integer
print("Multiply by 3:\n", x * 3)

# 4. Square
print("Square:\n", x**2)

# 5. Dot products
print("v.w:", np.dot(v,w))
print("x.v:", np.dot(x,v))
print("x.y:\n", np.dot(x,y))

# 6. Concatenation
print("Concat rows:\n", np.concatenate((x,y), axis=0))
print("Concat columns:\n", np.concatenate((v.reshape(-1,1), w.reshape(-1,1)), axis=1))

# 7. Concatenate x and v
try:
    print(np.concatenate((x,v)))
except Exception as e:
    print("Error:", e)

# Explanation:
# Error occurs because x is 2D and v is 1D.
# Dimensions must match for concatenation.
```

```
Addition:
 [[ 6  8]
 [10 13]]
Subtraction:
 [[-4 -4]
 [-4 -3]]
Multiply by 3:
 [[ 3  6]
 [ 9 15]]
Square:
 [[ 1  4]
 [ 9 25]]
v.w: 219
x.v: [29 77]
x.y:
 [[19 22]
```

```
 [50 58]]
Concat rows:
 [[1 2]
 [3 5]
 [5 6]
 [7 8]]
Concat columns:
 [[ 9 11]
 [10 12]]
Error: all the input arrays must have same number of dimensions, but the array at index 0 has 2 dimension(s) and the array at index 1 h
```

Problem - 4: Matrix Operations:

• For the following arrays: A = np.array([[3,4],[7,8]]) and B = np.array([[5,3],[2,1]]); Prove following with Numpy:

1. Prove A.A–1 = I.
2. Prove AB ≠ BA.
3. Prove (AB)

T = BTAT .

• Solve the following system of Linear equation using Inverse Methods.

2x – 3y + z = –1 x – y + 2z = –3 3x + y – z = 9

{Hint: First use Numpy array to represent the equation in Matrix form. Then Solve for: AX = B}

• Now: solve the above equation using np.linalg.inv function.{Explore more about "linalg" func- tion of Numpy}

```
A = np.array([[3,4],[7,8]])
B = np.array([[5,3],[2,1]])

# 1. Prove A.A^-1 = I
A_inv = np.linalg.inv(A)
print("A * A_inv:\n", np.dot(A, A_inv))

# 2. Prove AB ≠ BA
print("AB:\n", np.dot(A,B))
print("BA:\n", np.dot(B,A))

# 3. Prove (AB)^T = B^T A^T
left = np.dot(A,B).T
right = np.dot(B.T, A.T)
```

```
print("(AB)^T:\n", left)
print("B^T A^T:\n", right)
```

```
A * A_inv:
 [[1.00000000e+00 0.00000000e+00]
 [1.77635684e-15 1.00000000e+00]]
AB:
 [[23 13]
 [51 29]]
BA:
 [[36 44]
 [13 16]]
(AB)^T:
 [[23 51]
 [13 29]]
B^T A^T:
 [[23 51]
 [13 29]]
```

```
A = np.array([[2,-3,1],
              [1,-1,2],
              [3,1,-1]])

B = np.array([-1,-3,9])

# Using inverse
A_inv = np.linalg.inv(A)
X = np.dot(A_inv, B)

print("Solution using inverse:", X)

# Direct solve method (better method)
X2 = np.linalg.solve(A,B)
print("Solution using solve():", X2)
```

```
Solution using inverse: [ 2.  1. -2.]
Solution using solve(): [ 2.  1. -2.]
```

10.2 Experiment: How Fast is Numpy?

In this exercise, you will compare the performance and implementation of operations using plain Python lists (arrays) and NumPy arrays. Follow the instructions:

1. Element-wise Addition:

• Using Python Lists, perform element-wise addition of two lists of size 1, 000, 000. Measure and Print the time taken for this operation. • Using Numpy Arrays, Repeat the calculation and measure and print the time taken for this operation.

```
size = 1_000_000

# Python lists
list1 = list(range(size))
list2 = list(range(size))

start = time.time()
result = [a + b for a,b in zip(list1,list2)]
print("Python list addition time:", time.time() - start)

# NumPy arrays
arr1 = np.arange(size)
arr2 = np.arange(size)

start = time.time()
result = arr1 + arr2
print("NumPy addition time:", time.time() - start)
```

```
Python list addition time: 0.19423174858093262
NumPy addition time: 0.08937811851501465
```

2. Element-wise Multiplication

• Using Python Lists, perform element-wise multiplication of two lists of size 1, 000, 000. Measure and Print the time taken for this operation. • Using Numpy Arrays, Repeat the calculation and measure and print the time taken for this operation.

```
import time
start = time.time()
result = [a * b for a,b in zip(list1,list2)]
print("Python list multiplication time:", time.time() - start)

start = time.time()
result = arr1 * arr2
print("NumPy multiplication time:", time.time() - start)
```

```
Python list multiplication time: 0.12903094291687012
NumPy multiplication time: 0.025442838668823242
```

3. Dot Product

• Using Python Lists, compute the dot product of two lists of size 1, 000, 000. Measure and Print the time taken for this operation. • Using Numpy Arrays, Repeat the calculation and measure and print the time taken for this operation.

```
# Python
start = time.time()
dot = sum(a*b for a,b in zip(list1,list2))
print("Python dot product time:", time.time() - start)

# NumPy
start = time.time()
dot = np.dot(arr1,arr2)
print("NumPy dot product time:", time.time() - start)
```

```
Python dot product time: 0.1536712646484375
NumPy dot product time: 0.0028777122497558594
```

4. Matrix Multiplication

• Using Python lists, perform matrix multiplication of two matrices of size 1000x1000. Mea- sure and print the time taken for this operation.

• Using NumPy arrays, perform matrix multiplication of two matrices of size 1000x1000. Measure and print the time taken for this operation.

```
size = 1000

# Python matrix multiplication
A = [[1]*size for _ in range(size)]
B = [[1]*size for _ in range(size)]

start = time.time()
result = [[sum(A[i][k]*B[k][j] for k in range(size))
           for j in range(size)] for i in range(size)]
print("Python matrix multiplication time:", time.time() - start)

# NumPy matrix multiplication
A_np = np.ones((size,size))
```

```
B_np = np.ones((size,size))

start = time.time()
result = np.dot(A_np, B_np)
print("NumPy matrix multiplication time:", time.time() - start)
```

```
Python matrix multiplication time: 130.0278582572937
NumPy matrix multiplication time: 0.10162830352783203
```

Start coding or generate with AI.